به نام خدا

علی سپهرنیا (۹۶۱۲۱۰۳۸) گزارش کار آزمایشگاه سیستم دیجیتال ۲ (جلسه هشتم) استاد: خانم رضایی

۱- آزمایش ۱

با استفاده از واحد کپچر تایمر ۱ ، فرکانس موج مربعی ورودی میکرو اندازه گیری و در LCD نشان داده شود.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. به قسمت Timers > Timer1 میرویم. چون از کلاک داخلی استفاده می کنیم Clock Source را روی System Clock میگذاریم. Clock Value را روی 1000KHz تنظیم میکنیم و Mode را روی Normal میگذاریم. چون موج خروجی نمیخواهیم پس خروجی ها را برابر Disconnected قرار می دهیم. می خواهیم فرکانس موج وارد شده به واحد کپچر را اندازه گیری کنیم پس گزینه Input Capture: Rising Edge و Interrupt on: Input Capture را فعال میکنیم. برای استفاده از LCD گزینه Alphanumeric LCD را فعال میکنیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. ابتدا کتابخانه های موردنیاز stdlib و delay را اضافه میکنیم. چون رجیستر ICR1 تعریف نشده با دستور sfrw ICR1=0x26 آن را تعریف میکنیم. متغیر سراسری unsigned int period_us را تعریف میکنیم. تابع timer1_capt_isr به طور خودکار ایجاد می شود. این تابع به ازای هر پالس بالارونده که به واحد کپچر وارد میشود، فراخوانی می گردد. دستورات این تابع مطابق شکل پیوست، دوره تناوب را با کم کردن مقدار فعلی کپچر ICR1 از مقدار قبلی آن به دست می آورد و در period_us ذخیره میکند. برای تبدیل و نمایش اطلاعات تابع display را تعریف میکنیم. مطابق شکل پیوست در این تابع ابتدا period_us را به عنوان دوره تناوب روی LCD چاپ میکنیم. سپس از آنجایی که دوره تناوب برحسب میکروثانیه است مطابق فرمول $F = 1/T$ فرکانس می شود $1000000/T$. بعد از محاسبه F آن را تبدیل به رشته کرده و روی LCD چاپ میکنیم. داخل حلقه while بعد از یک delay_ms(500) تابع display را فراخوانی میکنیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که با دریافت موج وارد شده به واحد کپچر و کم کردن لبه ی بالای هر پالس از پالس قبلی، دوره تناوب و از روی آن فرکانس را محاسبه میکند و روی LCD نمایش میدهد. با آمدن هر پالس بالارونده، دوره تناوب و فرکانس بروزرسانی میشود.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. میکروکنترلر ATMEGA16، LCD، اسلیوسکوپ و منبع تولید پالس را به پروژه اضافه میکنیم. پایه های LCD را مطابق شکل پیوست به PORTA متصل میکنیم. منبع تولید پالس را هم به اسلیوسکوپ و هم به پایه PD6/ICP1 وصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که دوره تناوب و فرکانس روی LCD مطابق کانفیگ منبع تولید پالس تغییر میکند.

۲- آزمایش ۲

یک موج PWM با قابلیت تغییر فرکانس و دیوتی سایکل بسازید.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. به قسمت Timers > Timer1 میرویم. چون از کلاک داخلی استفاده می کنیم Clock Source را روی System Clock میگذاریم. Clock Value را روی 1000KHz تنظیم میکنیم. چون میخواهیم موج خروجی PWM با قابلیت تغییر بسازیم، Mode را روی Ph. & fr. Cor. PWM top=ICR1 میگذاریم. چون موج خروجی میخواهیم پس خروجی A را روی Non-Inverted PWM قرار می دهیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. چون رجیستر ICR1 تعریف نشده با دستور sfrw ICR1=0x26 آن را تعریف میکنیم. دیوتی سایکل را میتوانیم از طریق فرمول $DC = OCR1A/ICR1$ تنظیم کنیم. یعنی با تغییر مقدار OCR1A در حالی که ICR1 ثابت است میتوانیم دیوتی سایکل را تنظیم کنیم. همچنین با تغییر مقدار ICR1 در حالی که OCR1A ثابت است میتوانیم دوره تناوب و در نتیجه فرکانس را تغییر دهیم. مثلا برای دیوتی سایکل *75%* با فرض اینکه *ICR1=20000* باشد داریم:

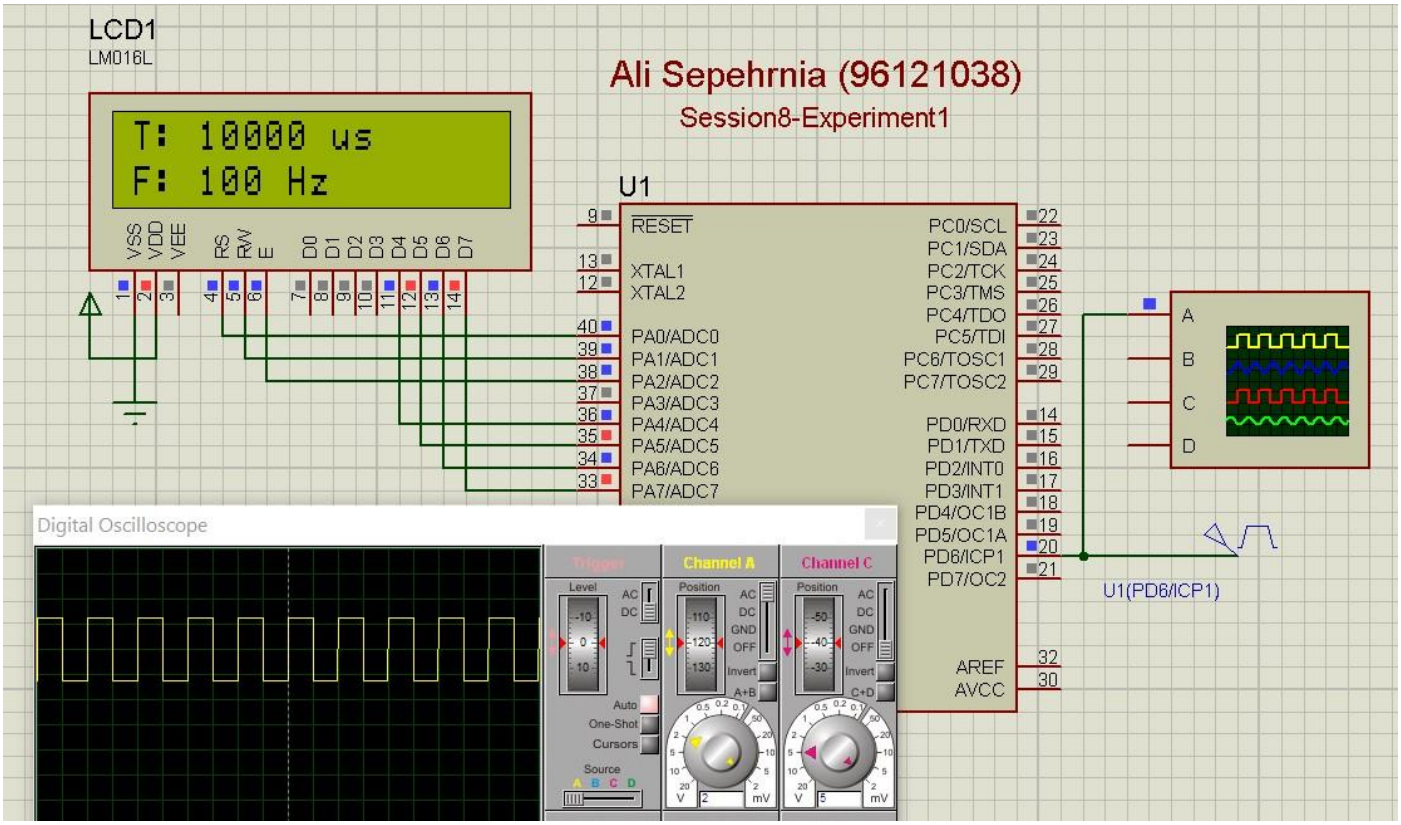
$$DC = \frac{OCR1A}{ICR1} \rightarrow \frac{3}{4} = \frac{OCR1A}{20000} \rightarrow OCR1A = 15000$$

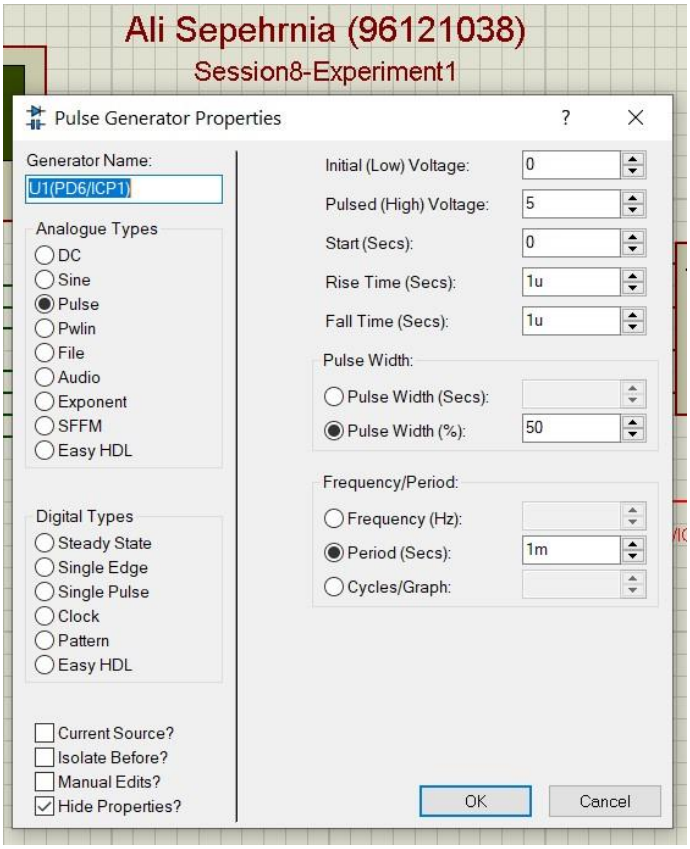
علاوه بر این در تایمر ۱ میتوان با ثابت نگه داشتن OCR1A و تغییر دادن ICR1 به فرکانس های مختلفی دست یافت.
برای تولید موج PWM با دیوتی سایکل 75% قبل از حلقه while مقدار ICR1 را برابر 20000 و در داخل حلقه مقدار OCR1A را برابر 15000 قرار میدهیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. میکروکنترلر ATMEGA16 و اسلیوسکوپ را به پروژه اضافه میکنیم. پایه PD5/OC1A که خروجی موج است را به اسلیوسکوپ متصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که موج PWM با دیوتی سایکل 75% تولید شده است.

پیوست ها:

آزمایش ۱)





G:\Personal\Uni\Micro Lab\Micro\Az8-1\Az8-1.c

Notes | Az8-1.c

```c
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session8-Experiment1
Version :
Date    : 11/11/2020
Author  : Ali Sepehrnia(96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <stdlib.h>
#include <delay.h>
// Alphanumeric LCD functions
#include <alcd.h>

// Declare your global variables here
sfrw ICR1=0x26;
unsigned int period_us;

// Timer1 input capture interrupt service routine
interrupt [TIM1_CAPT] void timer1_capt_isr(void)
```

G:\Personal\Uni\Micro Lab\Micro\Az8-1\Az8-1.c

Notes | Az8-1.c

```c
// Timer1 input capture interrupt service routine
interrupt [TIM1_CAPT] void timer1_capt_isr(void)
{
// Place your code here
    static unsigned int last_cap;
    period_us=ICR1-last_cap;
    last_cap=ICR1;
}

void display()
{
    char str[10];
    unsigned T,F;
    T=period_us;
    lcd_clear();
    lcd_puts("T: ");
    itoa(T,str);
    lcd_puts(str);
    lcd_puts(" us");
    ////////////////////////////
    lcd_gotoxy(0,1);
    lcd_puts("F: ");
    F=(unsigned long)1000000/T;
    itoa(F,str);
    lcd_puts(str);
    lcd_puts(" Hz");
}

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
```

G:\Personal\Uni\Micro Lab\Micro\Az8-1\Az8-1.c

Notes Az8-1.c

```
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Timer/Counter 1 initialization
// Clock source: System Clock
// Clock value: 1000.000 kHz
// Mode: Normal top=0xFFFF
// OC1A output: Disconnected
// OC1B output: Disconnected
// Noise Canceler: Off
// Input Capture on Rising Edge
// Timer Period: 65.536 ms
// Timer1 Overflow Interrupt: Off
// Input Capture Interrupt: On
// Compare A Match Interrupt: Off
// Compare B Match Interrupt: Off
TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
```

G:\Personal\Uni\Micro Lab\Micro\Az8-1\Az8-1.c

Notes Az8-1.c

```
TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
TCCR1B=(0<<ICNC1) | (1<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (1<<CS10);
TCNT1H=0x00;
TCNT1L=0x00;
ICR1H=0x00;
ICR1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;
OCR1BL=0x00;

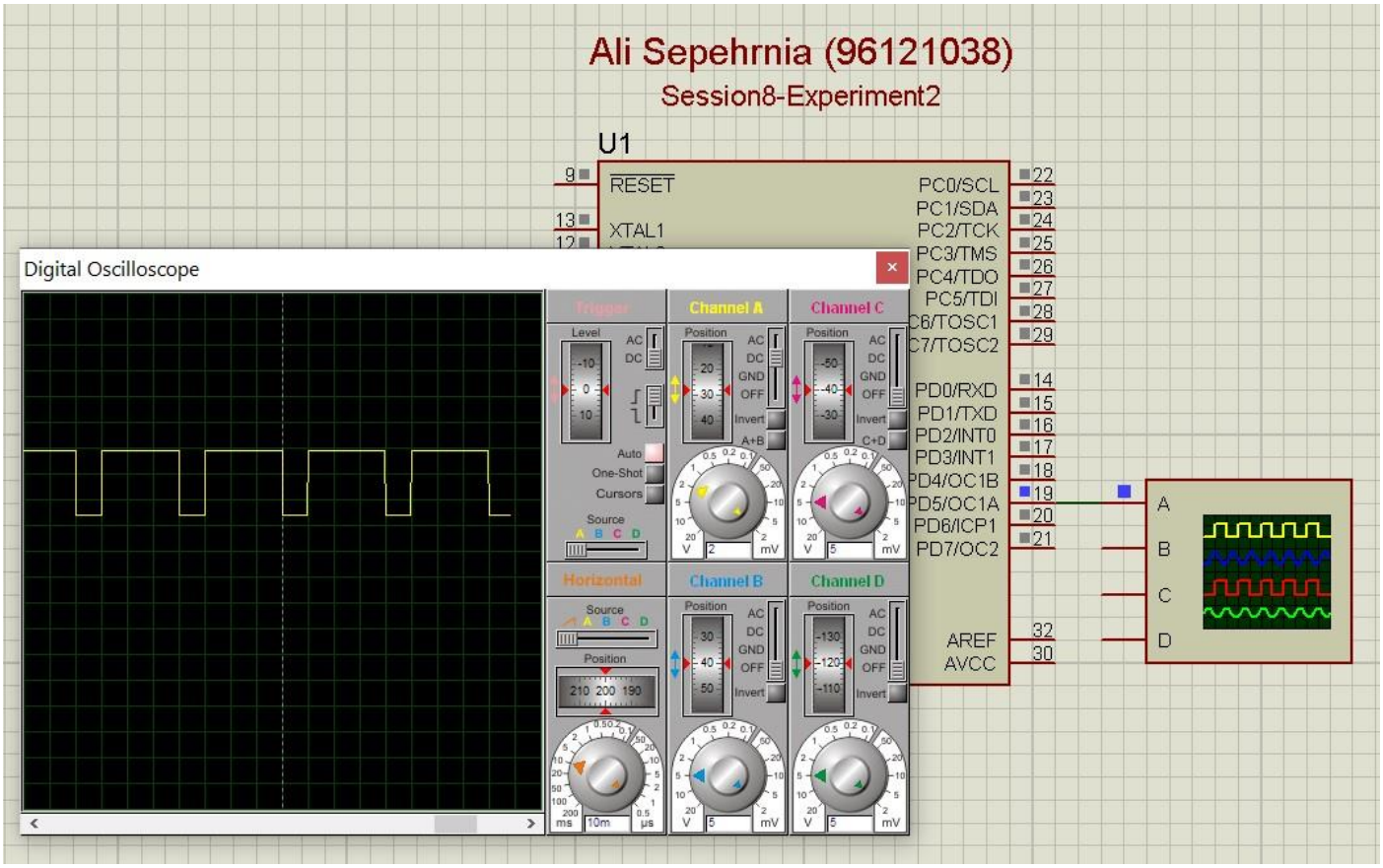
// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (1<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);

// Alphanumeric LCD initialization
// Connections are specified in the
// Project|Configure|C Compiler|Libraries|Alphanumeric LCD menu:
// RS - PORTA Bit 0
// RD - PORTA Bit 1
// EN - PORTA Bit 2
// D4 - PORTA Bit 4
// D5 - PORTA Bit 5
// D6 - PORTA Bit 6
// D7 - PORTA Bit 7
// Characters/line: 16
lcd_init(16);

// Global enable interrupts
#asm("sei")

while (1){
      // Place your code here
      delay_ms(500);
      display();
      }
}
```

آزمایش ۲)



G:\Personal\Uni\Micro Lab\Micro\Az8-2\Az8-2.c

Notes | Az8-2.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session8-Experiment2
Version :
Date    : 11/24/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
sfrw ICR1=0x26;

// Declare your global variables here

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
```

G:\Personal\Uni\Micro Lab\Micro\Az8-2\Az8-2.c

Notes | Az8-2.c

```
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=Out Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (1<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=0 Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Timer/Counter 1 initialization
// Clock source: System Clock
// Clock value: 1000.000 kHz
// Mode: Ph. & fr. cor. PWM top=ICR1
// OC1A output: Non-Inverted PWM
// OC1B output: Disconnected
// Noise Canceler: Off
// Input Capture on Falling Edge
// Timer Period: 0 us
// Output Pulse(s):
// OC1A Period: 0 us
// Timer1 Overflow Interrupt: Off
// Input Capture Interrupt: Off
```

G:\Personal\Uni\Micro Lab\Micro\Az8-2\Az8-2.c

Notes | Az8-2.c

```
// Compare A Match Interrupt: Off
// Compare B Match Interrupt: Off
TCCR1A=(1<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
TCCR1B=(0<<ICNC1) | (0<<ICES1) | (1<<WGM13) | (1<<WGM12) | (0<<CS12) | (0<<CS11) | (1<<CS10);
TCNT1H=0x00;
TCNT1L=0x00;
ICR1H=0x00;
ICR1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;
OCR1BL=0x00;

// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
ICR1=20000;

while (1){
      // Place your code here
        OCR1A=15000; //DC=OCR1A/ICR1
      }
}
```